Applications

Applications-1

1. Check if there exists a cycle in G.
2. Check if a graph G is a bipartite graph
3. Given a graph G, find the diameter of G.
4. Return the number of connected components in a graph.
5. Given two vertices u and v, check if the vertices are connected. As an extension, check if there exists a path from u to v, and from v to u.

BFS and DFS takes O(E) time if G is connected.

Applications-2

1. Check if the graph G is strongly connected. A graph G is strongly connected, if there exists a path from u to v, and from v to u for all pairs u,v.
2. Given a graph G, check if G is 2-edge-connected.
3. Given a graph G, check if G is 2-vertex-connected.
4. Check if a graph G is planar.

Cycle

Check if there exists a cycle in G.

Scenario: Imagine tasks in a project where some tasks depend on others.

Task A → Task B → Task C → Task D

If suddenly Task D depends on Task A, then we get:

A → B → C → D → A

This creates a cycle, meaning the project **cannot start** because every task is waiting for another.

Cycle

Directed graph – existence of a back edge?

If there is a back edge then there is a cycle.

Undirected graphs ?

BFS – edges in a partition, or at least edges between consecutive partitions.

Bipartite graph

A graph G is called a **bipartite graph** if the vertex set is partitioned into two disjoint sets X and Y such that

Every edge has one endpoint in X and the other endpoint in Y.

Bipartite graph

Job assignment

Group 1: Workers, Group 2: Jobs

Edges represent which worker can do which job.

Workers → Jobs: Alice → Job1, Bob → Job2, Charlie → Job1

No worker connects to another worker.

Other Scenarios: Job assignment systems, Matching groups (Dating apps: two groups matched), Student–project allocation

Bipartite graph

A graph G is called a **bipartite graph** if the vertex set is partitioned into two disjoint sets X and Y such that

Every edge has one endpoint in X and the other endpoint in Y.

Thm: G is bipartite iff every cycle of G is of even length.

Observation: Every odd cycle in a BFS tree contains two consecutive vertices from the same level.

Bipartite graph

BFS from some arbitrary vertex.

What happens if edges between vertices in a level does not exists? Is it a bipartite graph?

Bipartite graph

BFS from some arbitrary vertex.

What happens if edges between vertices in a level does not exists? Is it a bipartite graph? YES

Otherwise, there exists an edge between two vertices at the same level.

If G has an odd cycle, then G is not bipartite

Bipartite graph

BFS from some arbitrary vertex.

What happens if edges between vertices in a level does not exists? Is it a bipartite graph? YES

Otherwise, there exists an edge between two vertices at the same level.

If G has an odd cycle, then G is not bipartite

If all cycles in G are of even length then G is bipartite?

Diameter

Given two vertices u and v, dist(u,v) denotes the length of a shortest path from u to v. The **diameter** of G is the max{ dist(u,v) for all u,v}

Internet Network Delay

Nodes = routers
Edges = connections between routers

What is the maximum number of hops needed for data to travel between two routers?

Scenarios: Internet routing optimization
Network performance analysis,
Transportation / Metro System

Diameter

Given two vertices u and v, dist(u,v) denotes the length of a shortest path from u to v. The **diameter** of G is the max{ dist(u,v) for all u,v}

Procedure?

Relation between Diam(G) and BFS_Level?

Applications

1. Check if there exists a cycle in G.
2. Check if a graph G is a bipartite graph
3. Given a graph G, find the diameter of G.
4. **Return the number of connected components in a given graph.**
5. Given two vertices u and v, check if the vertices are connected. As an extension, check if there exists a path from u to v, and from v to u.

Number of Connected components

Social networks: Imagine users in a social media network.

Detecting isolated communities in social networks,
Identifying disconnected computer networks

PSEUDOCODE

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2       u.color = WHITE
 3       u.d = ∞
 4       u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11       u = DEQUEUE(Q)
12       for each vertex v in G.Adj[u]    // search the neighbors of u
13            if v.color == WHITE         // is v being discovered now?
14                 v.color = GRAY
15                 v.d = u.d + 1
16                 v.π = u
17                 ENQUEUE(Q, v)          // v is now on the frontier
18       u.color = BLACK                  // u is now behind the frontier
```

Number of Connected components

BFS from an unvisited vertex.

Store the component number of the vertex.

Repeat the above process until all vertices are visited.

How to find an unvisited vertex?

Running time?

Applications

1. Check if there exists a cycle in G.
2. Check if a graph G is a bipartite graph
3. Given a graph G, find the diameter of G.
4. Return the number of connected components in a given graph.
5. **Given two vertices u and v, check if the vertices are connected.** As an extension, check if there exists a directed path from u to v, and from v to u.

Connectivity

Internet routing
Nodes = computers/routers
Edges = network connections

If there is a path between Computer U and Computer V, they can communicate.

Scenarios: Checking network connectivity
Finding if two cities are reachable in a map

Connectivity

BFS from either u or v.

Directed path from u to v, and v to u.

DFS(G,u)

DFS(G,v)